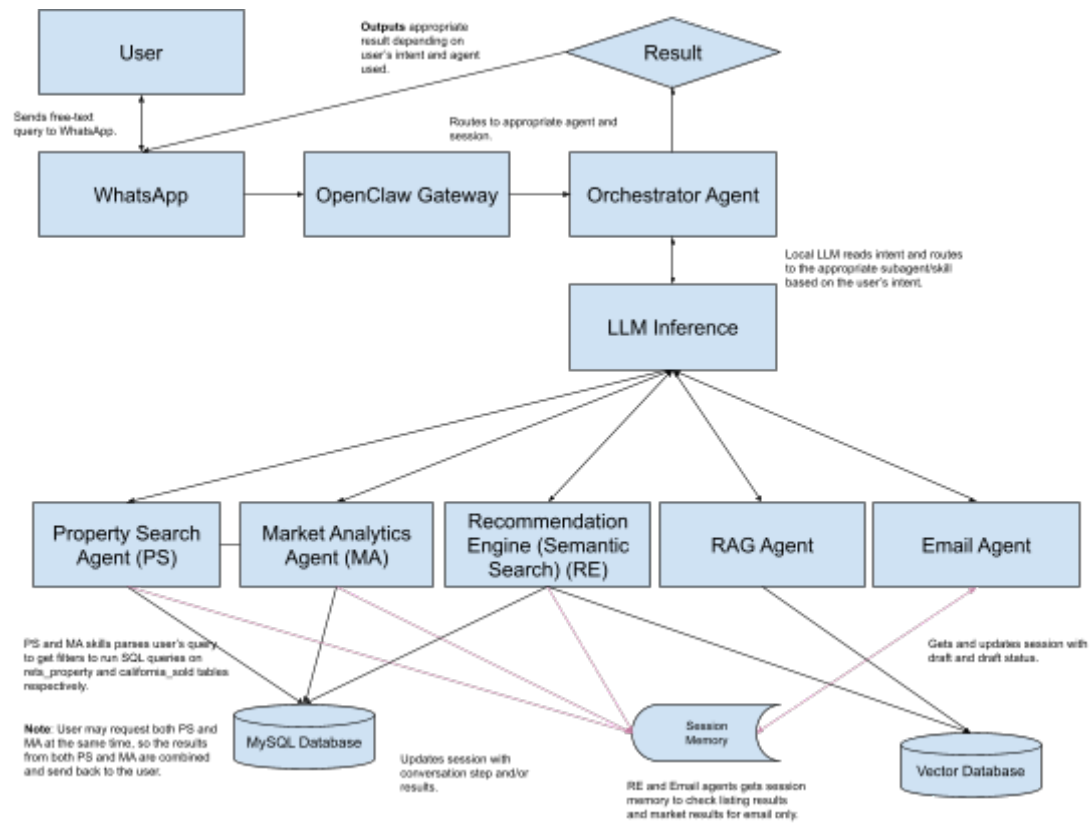


Architecture Diagram



Schema Annotation

rets_property (Active Listings)

Column Name	Data Type	Description	Usage Notes & Agent Logic
L_ListingID	VARCHAR	MLS system listing ID (Joins to california_sold.ListingKey)	Used by hybrid semantic search + recommendation engine to locate candidate listings
L_DisplayId	VARCHAR	Human-readable MLS number shown on portals	Retrieved from SQL query results
L_Address	VARCHAR	Street address	Shown in Property Search results
L_City	VARCHAR	City (Indexed)	Required filter parsed and used by Property Search Agent
L_Zip	VARCHAR	Postal code (Indexed)	Shown in Property Search results
L_SystemPrice	INT	Current listing price	Required filter parsed and used by Property Search Agent

L_Keyword2	INT	Total bedrooms	Required filter parsed and used by Property Search Agent
LM_Dec_3	DECIMAL	Total bathrooms (Supports half-baths, e.g., 2.5)	Filter parsed and used by Property Search Agent
LM_Int2_3	INT	Approximate finished square footage	Filter parsed and used by Property Search Agent
L_Type_	VARCHAR	Subtype: SingleFamilyResidence, Condominium, etc. (Indexed)	Required filter parsed and used by Property Search Agent
L_Status	VARCHAR	Listing status: Active, Pending, Withdrawn, etc.	Active status used in Property Search SQL query
LMD_MP_Latitude	DECIMAL	Geographic latitude (High precision)	Shown in Property Search results
LMD_MP_Longitude	DECIMAL	Geographic longitude (High precision)	Shown in Property Search results
YearBuilt	INT	Year property was constructed	Shown in Property Search results
AssociationFee	INT	Monthly HOA fee in dollars	Filter parsed and used by Property Search Agent

DaysOnMarket	INT	Days on market at time of data pull	Shown in Property Search results
PoolPrivateYN	VARCHAR	Private pool present (True/False)	Filter parsed and used by Property Search Agent
ViewYN	VARCHAR	Has a notable view (True/False)	Filter parsed and used by Property Search Agent
FireplaceYN	VARCHAR	Fireplace present (True/False)	Retrieved from SQL query results
PhotoCount	INT	Number of listing photos available	Shown in Property Search results
LA1_UserFirstName	VARCHAR	Listing Agent first name	Retrieved from SQL query results
LA1_UserLastName	VARCHAR	Listing Agent last name	Retrieved from SQL query results
LO1_OrganizationName	VARCHAR	Listing office / brokerage name	Retrieved from SQL query results

california_sold (Sold Listings)

Column Name	Data Type	Description	Usage Notes & Agent Logic

ListingKey	BIGINT	Unique listing identifier (Joins to rets_property.L_ListingID)	Retrieved from SQL query results from initial sold comp retrieval implementation
UnparsedAddress	VARCHAR	Full street address	Retrieved from SQL query results from initial sold comp retrieval implementation
City	VARCHAR	City of the property	Used by Market Analysis Agent to retrieve sold listings in specified city
CloseDate	VARCHAR	Date transaction closed (YYYY-MM-DD format)	Used by Market Analysis Agent to group and order analytic stats by month
ClosePrice	DOUBLE	Final sale/close price	Used by Market Analysis Agent to compute average price per month, list-to-close %, and average price/sq. ft.
OriginalListPrice	DOUBLE	Original asking price when first listed	Retrieved from SQL query results from initial sold comp retrieval implementation

ListPrice	DOUBLE	List price at time of contract	Used alongside ClosePrice to calculate list-to-close %
DaysOnMarket	BIGINT	Days from listing to contract	Used by Market Analysis Agent to compute average DOM per month
BedroomsTotal	DOUBLE	Total number of bedrooms	Retrieved from SQL query results from initial sold comp retrieval implementation
BathroomsTotalInteger	DOUBLE	Total number of bathrooms	Retrieved from SQL query results from initial sold comp retrieval implementation
LivingArea	DOUBLE	Finished living area in square feet	Used alongside ClosePrice to compute average price per square foot
PropertyType	VARCHAR	Residential, Land, ResidentialLease, CommercialSale, etc.	Used to validate comps in hybrid semantic search and recommendation engine agents
PropertySubType	VARCHAR	SingleFamilyResidence, Condominium, Duplex, etc.	Retrieved from SQL query results from initial sold comp retrieval implementation

YearBuilt	DOUBLE	Year property was built	Retrieved from SQL query results from initial sold comp retrieval implementation
ListAgentFullName	VARCHAR	Listing agent full name	Retrieved from SQL query results from initial sold comp retrieval implementation
ListOfficeName	VARCHAR	Listing brokerage name	Retrieved from SQL query results from initial sold comp retrieval implementation
BuyerOfficeName	VARCHAR	Buyer brokerage name	Retrieved from SQL query results from initial sold comp retrieval implementation

Written Reflection

What worked was using local LLMs over cloud LLMs. By using local LLMs, I can just simply run my agent/subagents anytime without worrying about any rate limits, reducing development and testing time on the agents. However, this came at the cost of inference time as I noticed that it takes more time for the agent to respond back than using a cloud LLM. Modifying the **AGENTS.md** and **SOUL.md** to get my local LLM to recognize natural language queries work out better than having to modify every individual **SKILL.md** file.

What could change was that I would stick to purely just one language, particularly with Python, since I am more familiar with Python than Typescript for the entire development process. I would have used Docker to OpenClaw to mitigate security issues as OpenClaw can execute commands and interact with the network with the potential to expose sensitive data. Alternatively, I would have run OpenClaw on a separate machine rather than just using my personal computer. Furthermore, I would have used another number instead of my personal number for WhatsApp to ensure my personal account doesn't get locked up as I ended once getting my personal account temporarily locked. Finally, I would have used a proper testing framework such as pytest rather than just using smoke testing when running through my safety guardrails test suite.